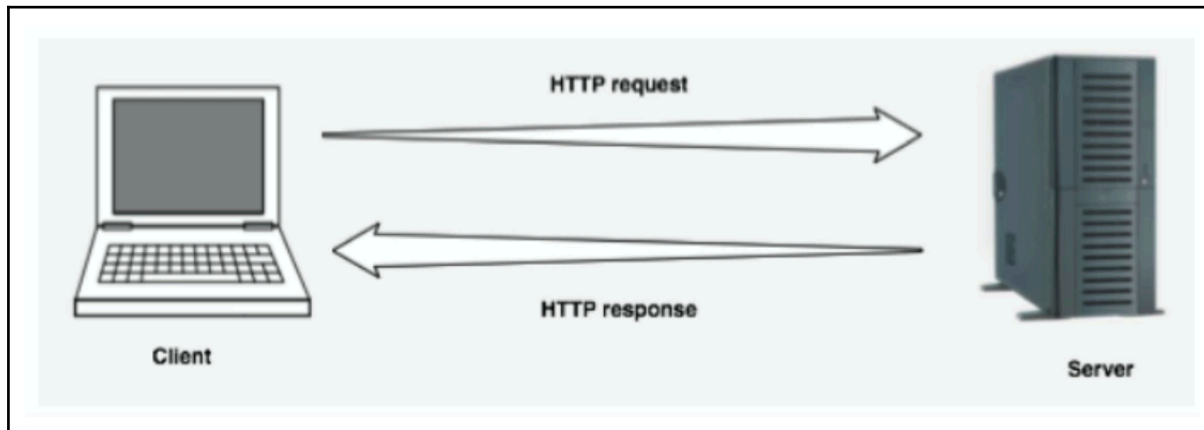


Section 5: Web Concepts in System Design

Web Sessions: Managing State in Web Apps

In this lecture we will explore how web applications maintain user state despite HTTPS being stateless and the mechanisms that make seamless user experience possible.



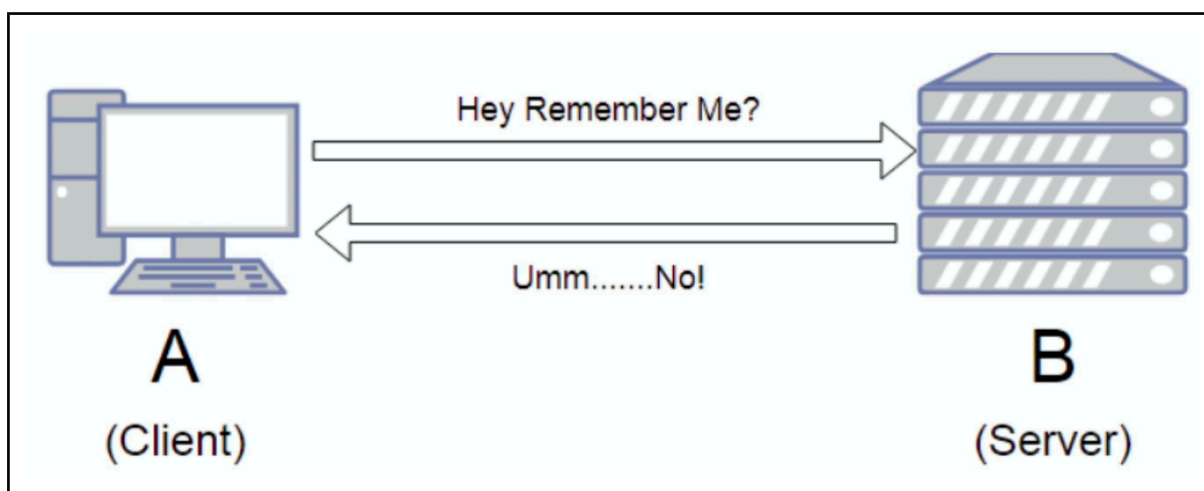
Why Web Sessions Matter?

Have you ever wondered how websites remember that you are logged in, keep items in your cart or save your preference as you move from page to page. The challenge is that HTTPS is fundamentally stateless, each request arrives at the server as an independent interaction with no built-in memory of the previous request. Without a mechanism to maintain state, users would have to authenticate repeatedly, shopping carts would be lost between pages, and every request would feel like it was coming from a completely new visit, that would make modern applications impractical to use. This is why session management is such a fundamental web concept, it provides a way to associate multiple requests with the same user and maintain contact across the entire interaction.

Technologies like cookies, server-side sessions, and authentication tokens enable applications to track user state while balancing security, scalability and user experience. So in this lecture we will explore how these mechanisms work and how architects design systems that maintain states efficiently in large scale web applications.

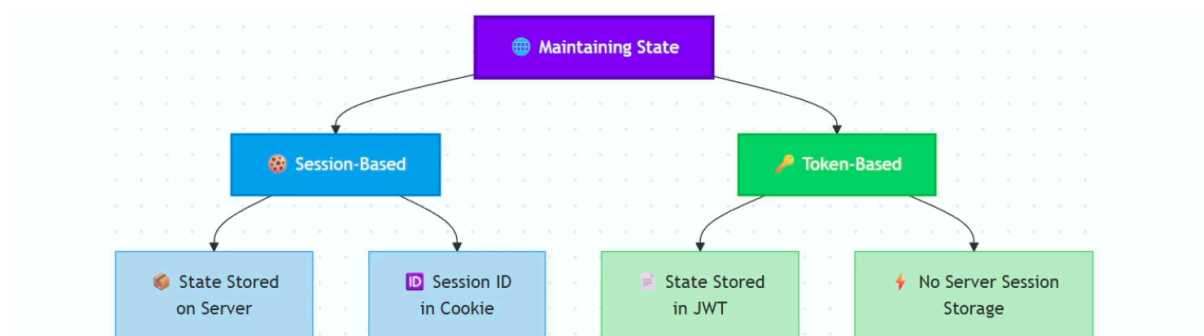
Understanding Statelessness in HTTP

HTTP being stateless is one of the reasons web scales so effectively, but it also creates an important challenge for developers. Every request that reaches the server is treated as a completely new interaction with no built-in memory of what happened before. Due to this each request must carry whatever context the server needs to process it, whether it's authentication information, session identifier or user specific data. The server cannot simply assume it remembers who you are from the previous request. The problem becomes very obvious when we think about real-world applications. After logging in you expect the website to recognize that you have access to dozens of pages, you expect your shopping cart to persist and your preference to remain available and none of that would be possible if the server truly forgets everything after each request. This is why session management exists.



Modern applications use mechanisms such as cookies, server-side sessions, and tokens to bridge the gap between HTTP stateless design and the stateful experience that people expect. Understanding this challenge is the foundation for understanding how sessions work.

Techniques for Maintaining States



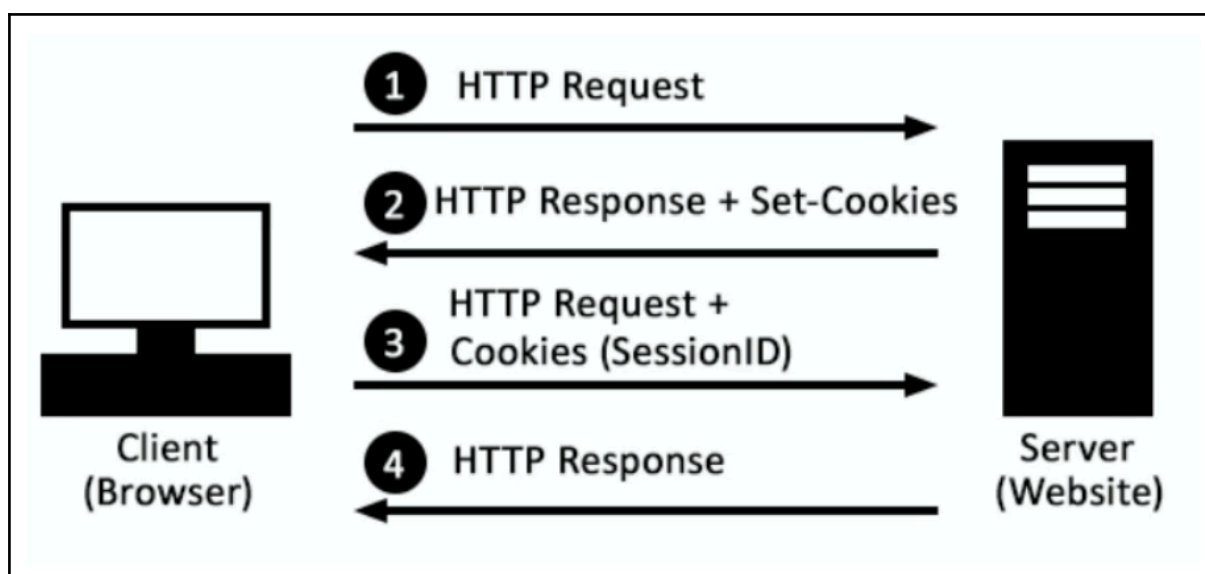
Since HTTPS doesn't itself remember previous requests, applications need a way to maintain user identity and context across multiple interactions. In practice there are 2 dominant approaches to solve this problem:

- **Session-Based Authentication:** In this technique the server owns the session state after the user logs in. The server creates a session record and returns the session identifier to the client, which is typically stored in a cookie on the client side. On subsequent request the client sends that session id and the server uses that id to retrieve the user session data. This model is straightforward and secure, but as the system grows managing and synchronizing session data across multiple servers can become an operational challenge.
- **Token-Based Authentication:** This technique uses a different approach, instead of storing session information on the server, the necessary user context is embedded inside a signed token such as JWT token and each request carries that token and the server validates it, without looking up any session state on the server side. This makes the architecture naturally stateless and easier to scale across distributed systems and microservices.

Neither approach is universally better, session-based authentication offers great control over user session while token-based authentication simplifies the horizontal scaling. As architects the choice depends on factors such as scalability requirements, security considerations, infrastructure complexity and how the application is expected to evolve over time.

Session-Based Authentication

Session-based authentication is one of the oldest and most widely adopted approaches for maintaining user states on the web. The key idea is simple, the server is responsible for storing session information while the client only stores a reference to that session.

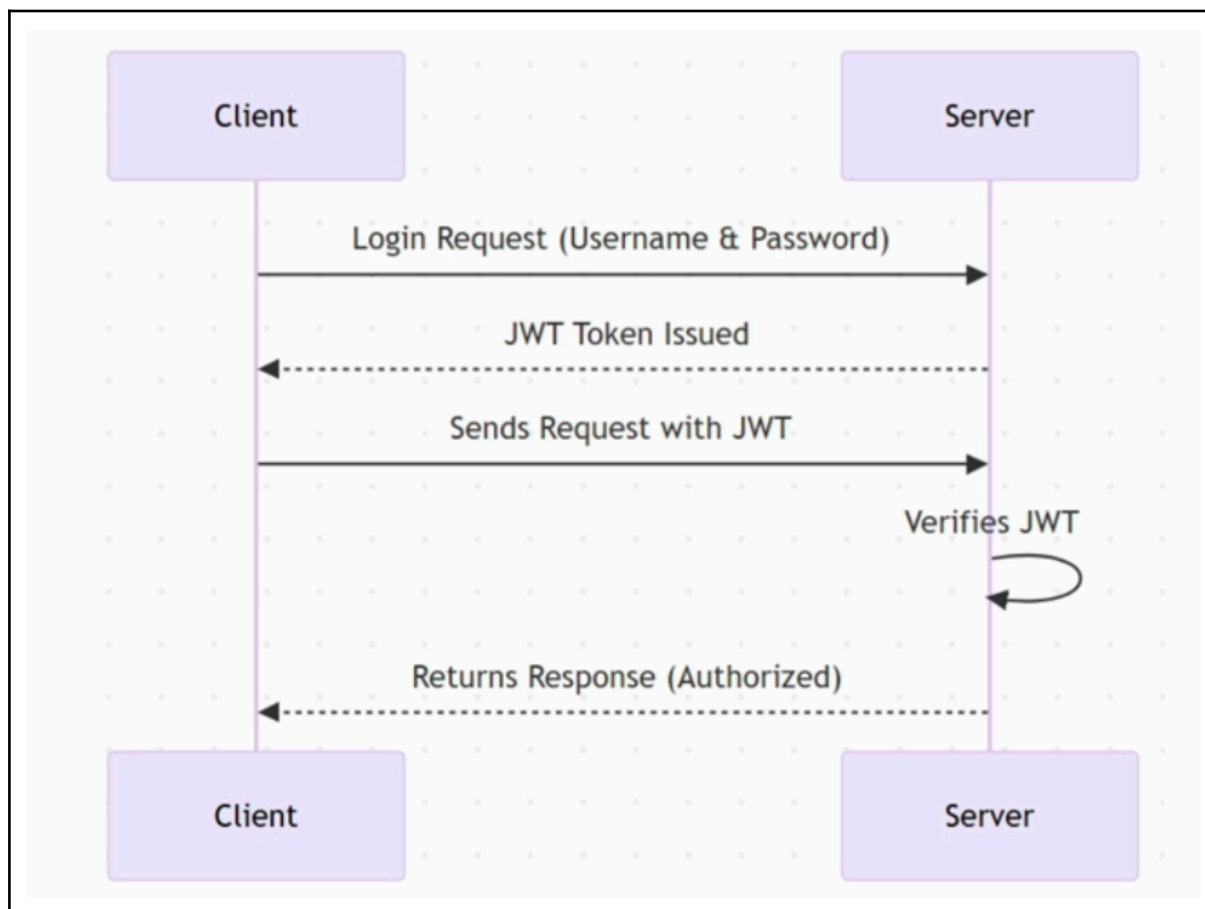


When a user successfully logs in, the server creates a session record containing information such as identity, permissions, and other relevant state. The server then generates a unique session id and sends it back to the browser typically through a cookie. From that point onward every request automatically includes the session id cookie, the server uses this identifier to locate the corresponding session data and determine who the user is without requiring them to authenticate again.

In other words the cookie acts like a claim ticket while the actual session information remains securely stored on the server. This model works extremely well for traditional web applications because it gives the server complete control over session management. However as applications scale across multiple servers, session storage becomes an architectural concern. To ensure any server can access session data, organisations often use techniques such as sticky sessions, session replication or centralized session store like redis. Understanding these trade offs is very important because session management decisions directly impact scalability, reliability and operational complexity.

Token -Based Authentication

Token-based authentication takes a fundamentally different approach to maintaining a user identity. Instead of storing session information on the server, the required user is packaged into a self-contained token that travels with every request.

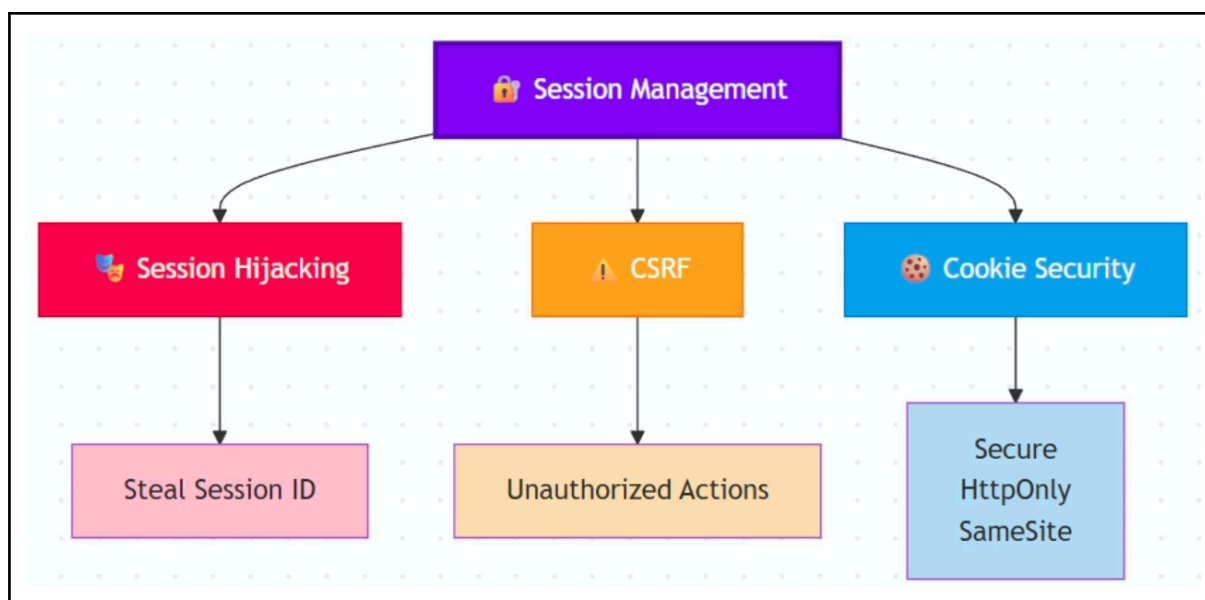


When a user authenticates, the server generates a token commonly a JWT token, which contains information such as user identity, rules and permissions. The client stores this token and sends it back with subsequent requests typically in authorization handling. Rather than looking up session data the server validates the token and extracts the information that it needs.

The biggest advantage of this model is scalability. Since the server doesn't maintain states any instance in a distributed system can process the request without relying on shared session storage. This makes token based authentication particularly attractive for APIs, microservices, mobile applications and cloud native architectures. However statelessness introduces its own trade-offs, once the token is issued, managing expiration, revocation and security becomes more challenging than simply deleting a server-side session. As a result token security, expiration policies and proper validation becomes a critical part of overall authentication design. And this is why modern systems often choose token-based authentication when scalability and service independencies are priority, while accepting the additional complexity that comes with managing the secure tokens.

Security Concerns in Session Management

Session management is not just about managing user state, it is also a critical security boundary. If an attacker can compromise a user session, they can often bypass the authentication entirely and act as that user. This is why securing session state is just as important as creating them.



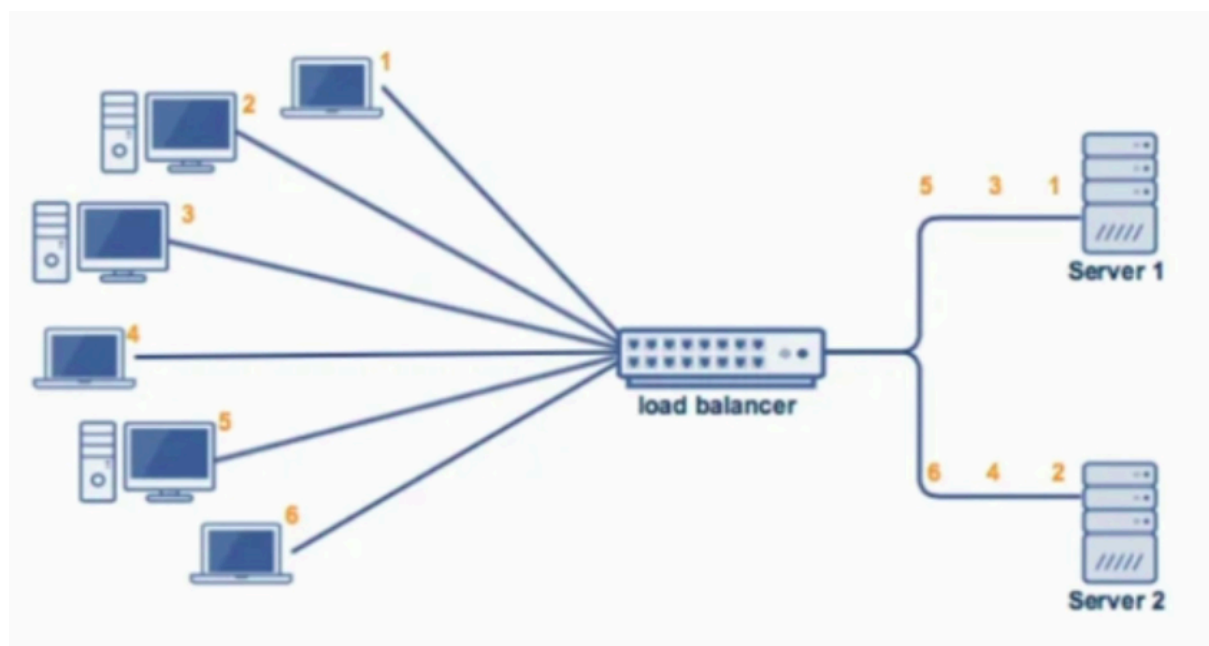
One of the most common threats is session hijacking where an attacker obtains a valid session id of another user, and uses it to impersonate that user. This can happen through techniques such as network interception, on unsecured connection or client side vulnerabilities like XSS injection. To reduce this risk, modern applications and HTTPS rotate session identifiers after authentication and limit session lifetime.

Another major concern is Cross-Site Request Forgery (CSRF). In this attack a malicious site tricks a user browser into sending authenticated requests to another application where the user is already logged in. Because the browser automatically includes session cookies, the request may appear legitimate even though they aren't. Defenses such as CSRF tokens, samesite cookies and additional verification for sensitive action help to prevent these attacks.

Finally cookies themselves need to be protected as well. Since cookies often carry authentication credentials or session identifiers, improperly configured cookies become an attractive target for attackers. Security flags such as Secure, HttpOnly and Samesite significantly reduce the risk of interception, script page access and cross-site abuse.

As an architect it is very important to remember that authentication is only the first step, the real challenge is protecting the session throughout its entire lifecycle: creation and usage to expiration and revocation.

Best Practices for Scaling Session Management



As traffic grows and applications move from a single server to distributed environments, session management becomes an architectural scaling challenge. A solution that works perfectly for one server can also quickly become a bottleneck when requests are distributed among dozens or even hundreds of instances. The below are some of the approaches used to scale session management:

- **Sticky Sessions:** In sticky sessions, a load balancer ensures that a user request always reaches the same server. While this is simple to implement, it creates uneven load and impacts availability that server becomes unhealthy.

- **Distributed Session Management:** Instead of storing sessions locally on application servers, session data is stored in the same storage all storage can access. This allows requests to be routed to any available instance improving both scalability and fault tolerance. Technologies like redis, memcache are commonly used for this purpose as they are in-memory data stores and they provide extremely fast session lookup while enabling session sharing across the entire server fleet. In large scale system redis is mostly referred because it offers additional capabilities such as persistence, replication and high availability.
- **Stateless Authentication:** For maximum scalability many modern architectures move towards stateless authentication using JWTs, since authentication data is carried within a token itself, application servers no longer need to perform session lookup. This reduces infrastructure dependencies and works particularly well in microservices, cloud native platforms and API-driven systems.

As architects the key lesson is that session management evolves with scalability, small applications may start with local sessions, growing systems often adopt distributed session stores and highly distributed platforms frequently embrace stateless authentication to minimize the operational complexity and improve the overall scalability.

Summary & Key Takeways

- HTTP is stateless, requiring session management techniques
- Cookies, server-side sessions, JWTs are common methods
- Security risks like session hijacking and CSRF must be mitigated
- Scaling sessions requires distributed storage solutions
- What's next:
 - Serialization: Data Exchange & Storage Formats

Important Interview Questions on Web Sessions

✓ Understanding Statelessness & State Management

- Why is HTTP considered a stateless protocol?
- How do web applications maintain state despite HTTP being stateless?

✓ Session Management Techniques

- What are the differences between server-side sessions and client-side tokens?
- When would you use JWT-based authentication instead of session-based authentication?
- How do cookies and sessions work together in session-based authentication?

✓ Security & Best Practices

- What is session hijacking, and how can it be prevented?
- How does CSRF exploit session management, and what are the mitigation strategies?
- Why should cookies be set with Secure, HttpOnly, and SameSite attributes?
- How can session management be scaled in distributed systems?

✓ Real-World Scenarios & System Design

- How does a load-balanced system handle session storage?
- In a microservices architecture, how do you manage authentication across multiple services?
- How do large-scale applications like Facebook, Amazon, or Google handle user sessions efficiently?